Mastering plan-based multi-agent systems requires building from simple sequential task lists up to self-healing Directed Acyclic Graphs (DAGs) with isolated worker contexts. Combining **LangGraph** for state management, checkpointing, and dynamic fan-out (Send and Command primitives) with **custom Python orchestration logic** (such as graphlib for dependency resolution) gives you both production durability and full control over scheduling mechanics.

Curriculum Roadmap

Stage	Focus Area	LangGraph Primitives	Custom Python Equivalent	Capstone Deliverable
Stage 1	**Linear Plan-and-Solve**	StateGraph, with_structured_output, Custom Reducers	collections.deque loop + Pydantic state container	Sequential research agent that generates, executes, and mutates a step list.
Stage 2	**Dynamic DAG & Parallel Fan-Out**	Send API, Map-Reduce patterns, Superstep synchronization	graphlib.TopologicalSorter + asyncio.gather	Dependency-aware DAG scheduler that dispatches independent nodes concurrently.
Stage 3	**Context Isolation & Subgraphs**	Compiled Subgraphs, Private Worker Schemas, State Mapping	Scoped worker classes receiving immutable state slices	Multi-domain system (SQL + Python + Search) with zero context leakage across workers.
Stage 4	**Critic Gates & Loop Dampening**	Command API (state update + routing), Negative Prompt Injection	Custom Retry/Circuit-Breaker decorator + Evaluator class	Self-correcting pipeline that catches hallucinations and replans without thrashing.

Stage 5	**Durability, HITL & Time-Travel**	AsyncPostgresSaver, interrupt(), State Forking / Replay	Append-only event log in PostgreSQL + state hydration	Production orchestrator with human plan approval and mid-graph crash recovery.

Stage-by-Stage Practical Guide

Stage 1: Linear Plan-and-Solve & State Isolation

Before introducing concurrency, build the foundational loop: **Plan** → **Execute Next Step** → **Update State** → **Replan or Finish**.

- **Core Mechanics:**
  - Define a deterministic Pydantic schema for your Plan (a list of Step objects containing id, description, assigned_worker, and status).
  - Separate **Global State** (the plan, completed results, and user objective) from **Scratchpad State** (the temporary tool-calling history of the active step).
- **LangGraph vs. Custom:**
  - *Custom:* A while plan_queue: loop that pops index 0, passes it to a worker function, appends the result to a results list, and passes results back to an LLM to optionally rewrite the remaining queue.
  - *LangGraph:* Use a StateGraph with three nodes: planner, executor, and replanner. Use a custom reducer function on your task list so completed tasks are updated in-place by id rather than blindly appended.
- **Hands-On Lab:** Build a technical due-diligence agent that takes a GitHub repository URL, plans 4 sequential audit steps (license check, dependency vulnerabilities, architecture summary, test coverage check), executes them one by one, and dynamically adds a step if a critical vulnerability is discovered.

Stage 2: Dynamic DAG Execution & Parallel Fan-Out

Linear plans bottleneck on latency. Real-world orchestration requires modeling tasks as a **Directed Acyclic Graph (DAG)** where Task C waits for Tasks A and B, while Tasks A and B run in parallel.

- **Core Mechanics:**
  - Upgrade your Task schema to include depends_on: list[str].
  - **The LangGraph Gotcha:** LangGraph's Send API dispatches parallel nodes within a *single superstep*, but it does not natively parse arbitrary multi-hop task dependencies inside your JSON plan. You bridge this by writing a **Custom Dispatcher Edge** in Python that evaluates which tasks have all depends_on IDs marked as completed, and emits Send(worker_node, task_payload) only for the currently unblocked frontier.
- **LangGraph vs. Custom:**
  - *Custom:* Python's built-in graphlib.TopologicalSorter. You call sorter.get_ready(), pass

those tasks to asyncio.gather(), and call sorter.done(*completed_ids) in a loop.
    - *LangGraph:* A central scheduler node or conditional edge that inspects state["tasks"], identifies the ready frontier, and returns a list of Send("worker", {"task": t}) objects. Once that parallel superstep finishes, all workers converge back to the scheduler via a state reducer.
- **Hands-On Lab:** Modify Stage 1 so the planner outputs a 6-node DAG. Verify with execution timestamps that independent branches (e.g., fetching API docs, querying a database, and scraping a changelog) execute in parallel within a single LangGraph superstep, while downstream synthesis waits until all upstream dependencies resolve.

Stage 3: Context Isolation & Hierarchical Subgraphs

A common failure mode in multi-agent systems is passing the entire global state (including every worker's raw tool outputs and stack traces) to every subsequent worker. This degrades reasoning accuracy and inflates token costs.

- **Core Mechanics:**
    - **Blackboard Pattern:** Workers never read the full graph state. The dispatcher resolves input references (e.g., injecting only the summary artifact of task_1 into the prompt of task_3) and passes a minimal WorkerInputState schema.
    - **Hierarchical Planning (Subgraphs):** When a worker's task is itself complex (e.g., "Write and debug a migration script"), encapsulate that worker as a compiled LangGraph subgraph with its own internal ReAct or Plan-Execute loop. The parent graph only sees the final verified artifact.
- **Hands-On Lab:** Create two specialized subgraphs: a CodeGeneratorSubgraph (Write → Pytest → Fix loop) and a DataAnalystSubgraph (Schema Inspect → SQL Query → Summarize loop). Wire both as worker nodes inside your Stage 2 parent DAG orchestrator, ensuring their internal message histories never enter the parent graph's state.

Stage 4: Critic Gates, Loop Dampening & Command Routing

Plans break when APIs time out, data is missing, or workers hallucinate. Your architecture needs deterministic verification gates and bounded replanning.

- **Core Mechanics:**
    - **Two-Tier Recovery:**
        1. *Local Retry:* If a worker fails a schema check or transient error, retry locally (up to N times) without invoking the heavy Planner LLM.
        2. *Global Replan:* If a task is fundamentally impossible (e.g., a required data source does not exist), route back to the replanner node to mutate the remaining DAG.
    - **Anti-Thrashing Guards:** Store a failed_plans: list[FailedAttempt] log in global state. Inject this log as explicit negative constraints into the replanner prompt so the LLM does not regenerate a previously failed path. Enforce a hard max_replans counter.
    - **LangGraph Command Primitive:** Instead of separating state mutation from conditional edge functions, return Command(update={...}, goto="replanner") directly from your critic node to update state and route execution atomically.
- **Hands-On Lab:** Inject a deliberate fault (e.g., a mock API tool that returns a 403

Forbidden on a specific endpoint, plus a secondary fallback tool). Build a critic node that catches the failure, records the broken path in failed_traces, and uses Command to route back to the replanner to reroute the DAG around the blocked tool.

Stage 5: Durability, HITL & Time-Travel Debugging

In production, a 15-step DAG cannot restart from scratch if Step 12 fails or requires human sign-off.

- **Core Mechanics:**
  - **Checkpointing:** Attach AsyncPostgresSaver to persist every superstep's state transition.
  - **Human-in-the-Loop (HITL) Plan Approval:** Use LangGraph's interrupt() inside a plan_review node right after the initial DAG is generated. Allow the user (or an API caller) to inspect the JSON DAG, edit task arguments, or prune nodes before resuming execution.
  - **Time-Travel / Forking:** Use graph.get_state_history(config) to locate a checkpoint right before a failed worker execution, patch the state payload via graph.update_state(), and resume execution from that exact node.
- **Hands-On Lab:** Wire your DAG orchestrator to a local PostgreSQL container using langgraph-checkpoint-postgres. Add an interrupt() gate before any destructive/external action node, inspect the serialized state, modify a task parameter mid-flight, and resume the graph to completion.

Starter Blueprint: DAG Orchestrator in LangGraph

Use this foundational skeleton for **Stages 1 & 2**. It combines Pydantic structured planning, custom dict-merging reducers, cycle validation via Python's graphlib, and parallel worker fan-out using LangGraph's Send API.

```
Python
import operator
from graphlib import TopologicalSorter, CycleError
from typing import Annotated, Literal, TypedDict
from pydantic import BaseModel, Field
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send, Command

# ==========================================
# 1. Schemas & State Reducers
# ==========================================

class TaskNode(BaseModel):
    id: str = Field(description="Unique task identifier, e.g., 'task_1'")
    worker: Literal["researcher", "coder", "analyst"]
```

```
    instruction: str
    depends_on: list[str] = Field(default_factory=list, description="IDs of prerequisite tasks")
    status: Literal["pending", "completed", "failed"] = "pending"
    result: str | None = None

class ExecutionPlan(BaseModel):
    tasks: list[TaskNode]

def merge_tasks(existing: dict[str, TaskNode], updates: dict[str, TaskNode]) -> dict[str, TaskNode]:
    """Custom reducer: merges parallel worker task updates into the central DAG dictionary."""
    merged = existing.copy() if existing else {}
    merged.update(updates)
    return merged

class OrchestratorState(TypedDict):
    objective: str
    tasks: Annotated[dict[str, TaskNode], merge_tasks]
    replan_count: int
    final_output: str | None

class WorkerPayload(TypedDict):
    """Scoped state passed to an isolated worker via Send()."""
    task: TaskNode
    upstream_context: dict[str, str]

# ==========================================
# 2. Graph Nodes
# ==========================================

async def planner_node(state: OrchestratorState) -> Command[Literal["scheduler"]]:
    """Generates a validated DAG from the user objective."""
    # Replace with: structured_llm = llm.with_structured_output(ExecutionPlan)
    # plan: ExecutionPlan = await structured_llm.ainvoke(state["objective"])

    # Example DAG: task_1 & task_2 run in parallel; task_3 depends on both
    mock_plan = ExecutionPlan(tasks=[
        TaskNode(id="task_1", worker="researcher", instruction="Fetch API specs", depends_on=[]),
        TaskNode(id="task_2", worker="analyst", instruction="Query DB schema", depends_on=[]),
        TaskNode(id="task_3", worker="coder", instruction="Write integration script", depends_on=["task_1", "task_2"])
    ])
```

```
    # Validate acyclicity using Python's standard graphlib before committing to state
    dag = {t.id: set(t.depends_on) for t in mock_plan.tasks}
    try:
        TopologicalSorter(dag).prepare()
    except CycleError as e:
        raise ValueError(f"Planner produced a cyclic dependency graph: {e}")

    task_map = {t.id: t for t in mock_plan.tasks}
    return Command(update={"tasks": task_map}, goto="scheduler")


async def scheduler_node(state: OrchestratorState) -> dict:
    """Synchronization barrier between parallel supersteps."""
    return {}


def route_ready_tasks(state: OrchestratorState) -> list[Send] | Literal["replanner", "synthesizer"]:
    """Evaluates DAG dependencies and fans out ready tasks in parallel via Send."""
    tasks = state["tasks"]

    # 1. Check if any task failed -> trigger replanner
    if any(t.status == "failed" for t in tasks.values()):
        if state.get("replan_count", 0) < 3:
            return "replanner"
        return "synthesizer"  # Graceful degradation if replan budget exhausted

    # 2. Check if all tasks are completed -> finish
    if all(t.status == "completed" for t in tasks.values()):
        return "synthesizer"

    # 3. Identify the unblocked frontier (pending tasks whose dependencies are all completed)
    completed_ids = {t_id for t_id, t in tasks.items() if t.status == "completed"}
    ready_tasks = [
        t for t in tasks.values()
        if t.status == "pending" and set(t.depends_on).issubset(completed_ids)
    ]

    # 4. Dispatch each ready task in parallel with ONLY its direct upstream results (Context Hygiene)
    dispatches = []
    for task in ready_tasks:
        scoped_context = {dep_id: tasks[dep_id].result for dep_id in task.depends_on}
        dispatches.append(
            Send("worker_executor", {"task": task, "upstream_context": scoped_context})
```

```
    return dispatches


async def worker_executor(payload: WorkerPayload) -> dict:
    """Executes a single bounded task in isolation and commits only its delta back to global state."""
    task = payload["task"]
    context = payload["upstream_context"]

    # Route to specialized subgraph or tool-calling agent based on task.worker
    updated_task = task.model_copy(
        update={
            "status": "completed",
            "result": f"[{task.worker.upper()} OUTPUT for {task.id} | Context keys: {list(context.keys())}]",
        }
    )
    return {"tasks": {task.id: updated_task}}


async def replanner_node(state: OrchestratorState) -> Command[Literal["scheduler"]]:
    """Mutates the remaining DAG when a worker node fails."""
    # Inspect failed tasks, rewrite downstream nodes, increment replan_count
    return Command(
        update={"replan_count": state.get("replan_count", 0) + 1},
        goto="scheduler",
    )


async def synthesizer_node(state: OrchestratorState) -> dict:
    """Aggregates terminal DAG nodes into the final response."""
    results = {t_id: t.result for t_id, t in state["tasks"].items()}
    return {"final_output": f"Completed {len(results)} tasks: {results}"}

# ==========================================
# 3. Graph Compilation
# ==========================================

builder = StateGraph(OrchestratorState)
builder.add_node("planner", planner_node)
builder.add_node("scheduler", scheduler_node)
builder.add_node("worker_executor", worker_executor)
builder.add_node("replanner", replanner_node)
builder.add_node("synthesizer", synthesizer_node)
```

```
builder.add_edge(START, "planner")
builder.add_conditional_edges("scheduler", route_ready_tasks, ["worker_executor", "replanner", "synthesizer"])
builder.add_edge("worker_executor", "scheduler")  # Converge parallel workers back to the scheduler barrier
builder.add_edge("synthesizer", END)

graph = builder.compile()
```

Recommended Study & Debugging Workflow

1. **Write the Pure-Python Engine First (2–3 hours):** Before expanding the LangGraph blueprint above, write a 60-line script using only asyncio.gather and graphlib.TopologicalSorter to execute the exact same ExecutionPlan Pydantic schema. Seeing how sorter.get_ready() and sorter.done() work in raw Python makes debugging LangGraph's superstep barriers effortless.
2. **Enforce Token Budgets in Tests:** Write a pytest fixture that asserts the payload size inside worker_executor. Ensure that upstream_context only contains immediate parent outputs (task.depends_on) and never leaks the global state dictionary.
3. **Trace Superstep Boundaries:** Run the compiled graph with graph.astream(..., stream_mode="updates"). Observe how task_1 and task_2 emit in the same parallel batch, hit the scheduler barrier, and then trigger task_3 in the next superstep.